Cooperative Coevolution for Large-Scale Optimization Based on Kernel Fuzzy Clustering and Variable Trust Region Methods

Jianchao Fan, *Member, IEEE*, Jun Wang, *Fellow, IEEE*, and Min Han, *Senior Member, IEEE*

***Abstract*—Large-scale optimization arises in a variety of scientific and engineering applications. In this paper, a particle swarm optimization (PSO) approach with dynamic neighborhood that is based on kernel fuzzy clustering and variable trust region methods (called FT-DNPSO) is proposed for large-scale optimization. The cooperative coevolution incorporated with a kernel fuzzy C-means clustering strategy is introduced to divide high-dimensional problems in to subproblems, and explore their search spaces. Furthermore, the independent variable ranges change adaptably by using the variable trust region learning method, which expedites the convergence process and explores in the effective space. In addition, the dynamic neighborhood topology assists the PSO algorithm in cooperating with neighbor particles and avoids the problem of premature convergence. Simulation results substantiate the effectiveness of the proposed algorithm to solve large-scale optimization problems with many well-known benchmark functions.**

***Index Terms*—Cooperative coevolution (CC), dynamic neighborhood topology, kernel fuzzy clustering, large scale optimization, particle swarm optimization (PSO), subswarms, trust region.**

I. Introduction

OVER the past decades, various metaheuristic optimization algorithms have been developed. For example, in recent years, differential evolution (DE) [1], [2], ant colony optimization (ACO) [3], [4], particle swarm optimization (PSO) [5]–[7], estimation of distribution algorithm (EDA) [8], and so on have emerged as popular representatives. PSO algorithms are based on a social psychological model of social influence and social learning. In PSO, every particle determines its velocity by using its previous velocity (momentum component), its best previous position (cognitive component), and its neighborhood best previous positions (social component). A population of candidate problem solutions, randomly initialized in a high-dimensional search space, discovers optimal regions of the space through a process of individuals' emulation of the successes of their neighbors [9]. Angeline [10] demonstrated the effectiveness of PSO in optimizing various continuous and discrete optimization problems. There are many applications of PSO, such as neural network training [7], PID controller tuning [11], power load distribution [12], multicriteria fuzzy decision making [13], and so on, with good results.

Despite many success stories, classical evolutionary algorithms often lose their efficacy and advantages when they are applied to large and complex problems such as those with high dimensions of search space. Their performance deteriorates rapidly as the dimensionality of the search space increases [14]. The reason for this phenomenon is not only that there are local optimal solutions, but also that the velocities of particles sometimes might lapse into the degeneracе so that the successive range is restricted in the subplain of the whole search hyperplain [15]. Furthermore, there are many black-box objective functions in actual industrial processes [16]. Hence, finding a quick, efficient, and derivative-free optimization approach to tackle the high-dimensional optimization problem is both necessary and rewarding.

This issue has actually been addressed in recent years. Korenaga [17] used a coordinate conversion method to rotate particle swarm. Although such an improvement works well when dealing with unimodal functions, other complex problems could not yield a better solution. Bergh *et al.* [18] first proposed a cooperative approach for PSO algorithms. Yang *et al.* [14] adopted a random grouping scheme and adaptive weighting in problem decomposition and coevolution, termed DECC-G. A dynamic multiswarm particle swarm optimizer with local search method was introduced by Suganthan, which balanced the local and global searching ability well [19]. Subbaraj [20] presented an advanced parallelized PSO algorithm with modified stochastic acceleration factors to solve large-scale economic dispatch problems with prohibited operating zones, ramp-rate limits, and transmission losses. Omidvar [21], [22] drew the conclusion, based on [14], that the probability of grouping interacting variables in one subcomponent using random grouping drops significantly as the number of interacting variables increases. Therefore, more frequent random grouping [21] and delta grouping [22] strategies were employed to divide more interacting variables into one group. The delta method measures the averaged difference in a certain variable across the entire

Manuscript received March 19, 2013; revised May 14, 2013 and June 23, 2013; accepted July 1, 2013. Date of publication August 7, 2013; date of current version July 31, 2014. This work was supported in part by the Project 61273307 of the National Nature Science Foundation of China, in part by the Foundation (201313) of Key Laboratory of Marine Spill Oil Identification and Damage Assessment Technology, and in part by the Research Grants Council of the Hong Kong Special Administrative Region, China, under Grant CUHK416812E.

J. Fan is with the Department of Ocean Remote Sensing, National Marine Environmental Monitoring Center, Dalian 116023, China (e-mail: jcfan@nmemc.gov.cn).

J. Wang is with the School of Control Science and Engineering, Dalian University of Technology, Dalian 116024, China and also with the Department of Mechanical and Automation Engineering, The Chinese University of Hong Kong, Shatin, New Territories, Hong Kong (e-mail: jwang@mae.cuhk.edu.hk).

M. Han is with the School of Control Science and Engineering, Dalian University of Technology, Dalian 116024, China (e-mail: minhan@dlut.edu.cn).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TFUZZ.2013.2276863

population and uses it to identify interacting variables. Based on [19], Suganthan utilized the subregion harmony search in [23] and constraint handling techniques in [24] to enhance optimization capability.

However, because of the characteristics of high-dimensional optimization, the particles following the best particle in the whole population would converge too fast to obtain extreme points [25]. In view of this, Kennedy [26] provided several social network structures testing to verify the significant effect on different neighborhood topologies performance. Liu [27] presented the neighborhood topology particle swarm optimization in the optimal design of a cylinder linear induction motor. Chen [28] proposed a new linkage identification technique called dynamic linkage discovery to address the linkage problem in real-parameter optimization problems. Most linkages are the static and invariable relationship, such as random topology or the circles [27], [29]–[32]. In addition, most large-scale optimization issues were solved through the cooperative coevolution (CC) [14], [21], [22] or multiswarms approach [19], [23], [33], whose structures are simple and random or fixed only.

To address the aforementioned drawbacks a dynamic neighborhood PSO algorithm with the kernel fuzzy clustering and trust region method (FT-DNPSO) is proposed. The main work of this paper is as follows.

1) The trust region method is utilized to adjust search range adaptively, which could expedite the optimization process. When the next iterative point $x(k+1)$ makes the objective function $f(x)$ decrease substantially, the trust region Δ_k (independent variable range) turns narrow. It makes particles quickly look for optimal points, because there is no need to waste iterative times to search in the inefficacy space.
2) The grouping is a key issue in the cooperative coevolution; how many dimensions should be assigned to one group? Which dimension should be grouped into the same subproblem? To address these issues, a kernel fuzzy C-means (KFCM) approach is employed to analyze variable relations, and gather similar ones into the same group.
3) Dynamic neighborhood topology strategy is adopted in FT-DNPSO. The adaptive topology strategy could learn from the neighborhood positions, and constitute the subswarm self-organized to optimize it and share the information, which could achieve better precision and results.

The rest of this paper is organized as follows. Section II presents the basic PSO algorithm and classic topology structure. Section III describes the modified PSO version termed as for the optimization of high-dimensional problems. Section IV highlights the potential of the proposed approach by use of experimental results. Concluding remarks are given in Section V.

II. Preliminaries

A. Particle Swarm Optimization

PSO [5] is a swarm-based intelligent optimization algorithm, which maintains a swarm of candidate solutions, referred to as "particles." Denote the best solution of a particle achieved so far as $P_{\{i,\text{best}\}} = (p_{i1}, p_{i2}, \ldots, p_{iD})$. The best position among all the particles in the population is represented as $P_{g\text{best}} = (p_{g1}, p_{g2}, \ldots, p_{gD})$ and D is the dimension of space. PSO relies on the exchange of information between particles, in other words, this is a hypothesis that social sharing of information among individuals offers an evolutionary advantage. PSO is similar to other evolutionary algorithms in that the system is initialized with a population of random solutions. The velocity for each particle is dynamically adjusted according to the flying experiences of its own and fellows. Therefore, it finds the global best solution by simply adjusting the trajectory of each individual toward its own best position and toward the best neighboring particle at each time step. The velocity v_{id} and position x_{id} update equations for the dth dimension of the ith particle in the swarm is denoted as

$$\begin{cases} v_{id}(k+1) = wv_{id}(k) + c_1\,\text{rand}_{1\text{d}}()[p_{id}(k) - x_{id}(k)] \\ \quad +c_2\,\text{rand}_{2\text{d}}()[p_{gd}(k) - x_{id}(k)] \\ x(k+1) = x(k) + v(k+1) \end{cases} \tag{1}$$

where $w \in [0,1]$ is the inertia weight, determining how much of the previous velocity of the particle is preserved. And $\text{rand}_1()$ and $\text{rand}_2() \in [0,1]$ denote two uniform random number samples, c_1 and c_2 are acceleration constants, and k denotes the iterative time. In a D-dimensional search space, the position vector of the ith particle is represented as $x_i = \{x_{i1}, x_{i2,} \ldots, x_{iD}\}$, where $x_{id} \in [x_{\min,\text{d}}, x_{\max,\text{d}}]$, $d = 1, \ldots, D$. $x_{\min,\text{d}}$ and $x_{\max,\text{d}}$ are the lower and upper bounds for the dimension d, respectively. Let $V_i = (v_{i1}, v_{i2}, \ldots, v_{iD})$ be the current velocity for particle i, which is limited to a maximum velocity $V_{\max} = (v_{\max,1}, \ldots, v_{\max,D})$. Positions and velocities are adjusted and the objective function to be optimized $f(x)$ is evaluated with the new coordinates. From the velocity update equation in (1), it is apparent that the second part represents the private thinking by itself, and the third part is the social part, which represents the cooperation among the individuals.

B. Classic Neighborhood Topology

There are several common neighborhood generating approaches. The first is that the set of particle connections is randomly generated [34]. Another way is the nearest neighbors in the search space. In this method, every particle is assumed to have a perception area and could only communicate with the particles which are within that area. Assuming that the radius of the perception area of particle i is given by $\zeta > 0$, a mathematical expression to determine its neighborhood at time k is presented as

$$\text{N}_i(k) = \{j, j \neq i |\, \|x_i(k) - x_j(k)\| \leq \zeta_i\} \tag{2}$$

where $\text{N}_i(k)$ is the set of neighbors of particle i at time k. On the other hand, the neighborhood relations could also be determined based on the distances in fitness function as

$$\text{N}_i(k) = \{j, j \neq i |\, \|f(x_i(k)) - f(x_j(k))\| \leq \zeta_i\} \tag{3}$$

where $f(\cdot)$ is the function being optimized. Another common topology is generated according to the probability statistics. Then, the neighborhood of the particle i is determined according

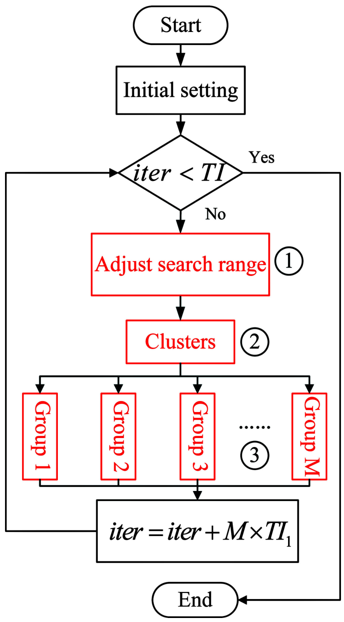


Fig. 1. FT-DNPSO schematic diagram.

to

$$\mathrm{N}_i(k) = \{j, j \neq i | \vartheta_{ij}^{\text{random}} \leq \vartheta\} \tag{4}$$

where $\vartheta_{ij}^{\text{random}}$ is a random probability between particle i and j, ϑ denotes the given probability of the neighborhood generation in the whole swarm. When $\vartheta_{ij}^{\text{random}} \leq \vartheta$, it means the particle i can obtain information from the particle j at time k.

The PSO algorithm has different kinds of topology structure, such as star, ring, four clusters, or Von Neumann structures [35]. A particle in a PSO with a different structure has different number of particles in its neighborhood with a different scope. The ring structure has the longest diameter among the aforementioned four special structures. A particle is connected with two neighbors in this topology, which needs only one step to two nearby neighbors, and another step to another two particles, and so on.

III. Cooperative Coevolution Framework Based on Fuzzy Clustering, Trust Region Method, and Dynamic Neighbor Particle Swarm Optimization

A. Framework

The popular strategies for large-scale optimization can be summarized as follows: problem decomposition, subcomponent optimization, and subcomponents coadaptation [14]. However, most actual problems cannot be decomposed. As a result, for an indivisible optimization problem, the flowchart of this paper proposed FT-DNPSO is shown in Fig. 1.

First, initialize swarm size m, total number TI of iterative times, search range $[x_{\min}, x_{\max}]$, and iterative TI_1 of each group. In the iterative process, the adaptively adjusting search range part, grouping part, metaheuristic optimization algorithm in each group are three main distribution work, which are presented in the following sections.

B. Variable Trust Region Method for Adjusting Search Range

The trust region method for unconstrained optimization problem has the most obvious global convergence advantage, which is different from the other conventional optimization methods such as steepest descent method, Newton method, and conjugate gradient method [36]. In this section, the similar trust region (TR) concept is introduced as

$$\Delta_k \in [x_{\min}, x_{\max}] \tag{5}$$

where Δ_k presents the trust region, which means the search range of particles. The main principle is as follows: define a model function $m_k(x)$, and expect it to approximate the objective function $f(x)$, at each best position $x_{\text{best}}(k)$.

If the optimal best heuristics step $v_{\text{best}}(k+1)$ could make $f(x)$ decrease substantially, reduce the trust region Δ_k to accelerate the convergence procedure. Some hints are needed to illustrated, such as $P_{g\text{best}}(k) = f(x_{\text{best}}(k))$, and $x_{\text{best}}(k)$ is the best position at the kth iteration; $P_{g\text{best}}(k+1) = f(x_*(k) + v_{\text{best}}(k+1))$, and $x_*(k)$ presents the position of the particle obtains the best update velocity at the next iteration. The model function $m_k(x)$ is defined as

$$m_k(x(k) + v(k+1)) = f(x(k)) + g_k^{\mathrm{T}} v(k+1) \tag{6}$$

where $g_k = \nabla f(x(k)) \approx \frac{f(x(k)) - f(x(k-1))}{x(k) - x(k-1)}$, and $v(k+1)$ is the particles' velocity as the next heuristics step. The stretch factor is defined as $\alpha(x) = p\exp(-qx^2)$, where $p \in (0,1), q > 0$. Hence, the pseudocode of *part* ① (see Fig. 1) is as follows:

Step 1: Give the constant variable p, q, and define the model function $m_k(\cdot)$ according to (6).

Step 2: At each best iterative point $x_{\text{best}}(k)$, calculate the $v_{\text{best}}(k+1)$ whether makes the objective function $f(x)$ to decrease substantially, which means the objective function getting better. Let

$$\rho_k = \frac{f(x_{\text{best}}(k)) - f(x_*(k) + v_{\text{best}}(k+1))}{m_k(x_{\text{best}}(k)) - m_k(x_{\text{best}}(k) + v_{\text{best}}(k+1))}. \tag{7}$$

Step 3: Then, the stretch factor is as follows:

$$\alpha(\rho_k) = p\exp(-q\rho_k^2) \tag{8}$$

and the variable trust region is denoted as

$$\Delta_{k+1} = \alpha(\rho_k)\Delta_k. \tag{9}$$

The stretch factor is defined as exponential function form, which could limit the variable range to (0, 1]. In addition, when the next particles' velocity $v_{\text{best}}(k+1)$ makes $f(x)$ decrease substantially, that means ρ_k becomes larger according to (7), trust region Δ_k becomes smaller in a great extent according to (8) and (9). The research range turns narrow adaptively, otherwise it makes few change. The proposed FT-DNPSO algorithm can go through the exploration research in a more effective region, which improves the convergence speed to a great extent.

C. Kernel Fuzzy C-Means Clusters for Cooperative Coevolution

In the rest of this section, we describe the KFCM algorithm from the mathematical view for cooperative coevolution, which is different from other PSO algorithms with the clustering modification. In the FT-DNPSO scheme, KFCM clusters the dimensions of problems in *part* ② and particles in *part* ③, respectively (see Fig. 1). Let $\mathbf{a}_i = \{a_{i,1}, a_{i,2}, \ldots, a_{i,N}\}^{\mathrm{T}} \in \Re^N$ be an unlabeled dataset, where $i = 1, 2, \ldots, D, N$ is the total number of particles, and D is the dimension of optimization problem. Fuzzy C-mean (FCM) is an optimization-based clustering algorithm, which divides the dataset into C $(C \in 2, \ldots, D-1)$ clusters represented as fuzzy sets $\{T_1, T_2, \ldots, T_C\}$, so as to minimize the following objective function:

$$J_m(U, B) = \sum_{j=1}^{D} \sum_{i=1}^{C} u_{ij}^m \left\| \Phi(\mathbf{a}_j) - \Phi(\mathbf{b}_i) \right\|^2 \tag{10}$$

where u_{ij} is the membership degree of data $\mathbf{a}_j$ to the clustering center $\mathbf{b}_i$, and is an element of a membership matrix $U = [u_{ij}]_{C \times D}$; m is a weighting exponent controlling the amount of clustering fuzziness, whose value is often determined by user; $B = \{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_C\}$ is a vector comprised of the centroid of fuzzy sets $\{T_1, T_2, \ldots, T_C\}$. Thus, a fuzzy partition can be denoted by the pair (U, B). The squared distance is computed in the kernel space using a kernel function such that [37]

$$\left\| \Phi(\mathbf{a}_j) - \Phi(\mathbf{b}_i) \right\|^2 = \mathrm{K}(\mathbf{a}_j, \mathbf{a}_j) + \mathrm{K}(\mathbf{b}_i, \mathbf{b}_i) - 2\mathrm{K}(\mathbf{a}_j, \mathbf{b}_i) \tag{11}$$

where $\mathrm{K}(\mathbf{a}_j, \mathbf{a}_j) = \Phi(\mathbf{a}_j)^{\mathrm{T}} \Phi(\mathbf{a}_j)$, the others are similar. $\Phi(\cdot)$ denotes an arbitrary nonlinear mapping from the original low-dimensional feature space to a dimension of higher dimensionality (kernel space). The kernel method takes advantage of the fact that the squared distance does not products in the kernel space, can be expressed by a kernel function $\mathrm{K}(\cdot)$, according to (11). Commonly encountered kernel functions are Gaussian, Polynomial, and Hypertangent. The Gaussian kernel function is adopted in our proposed approach. Then, the KFCM algorithm focuses on minimizing $J_m(U, B)$, subject to the following constraints on U

$$\sum_{j=1}^{D} u_{ij} > 0, \; \sum_{i=1}^{C} u_{ij} = 1, \; 0 \le u_{ij} \le 1 \;\; i = 1, 2, \ldots, C$$
$$j = 1, 2, \ldots, D \tag{12}$$

Equations (11) and (12) describe a constrained optimization problem, which can be converted to an unconstrained optimization problem by using the Lagrange multiplier technique. A solution to the objective function can be obtained via an iterative process where the degrees of membership u_{ij} and the clustering centers $\mathbf{b}_i$ are updated via

$$u_{ij} = \frac{1}{\sum_{t=1}^{C} \left(\frac{|\Phi(\mathbf{a}_j) - \Phi(\mathbf{b}_i)|^2}{|\Phi(\mathbf{a}_j) - \Phi(\mathbf{b}_t)|^2} \right)^{\frac{1}{m-1}}}$$
$$i = 1, \ldots, C; \quad j = 1, \ldots, N \tag{13}$$

$$\mathbf{b}_i = \frac{\sum_{j=1}^{N} (u_{ij})^m \mathbf{K}(\mathbf{x}_j, \mathbf{b}_i) \mathbf{x}_j}{\sum_{j=1}^{N} (u_{ij})^m \mathbf{K}(\mathbf{x}_j, \mathbf{b}_i)} i = 1, \ldots, C. \tag{14}$$

At the beginning of the KFCM algorithm, the clustering centers are initialized randomly. Then, the membership matrix is calculated and the clustering centers are updated using (13) and (14), respectively, in the iterative process, as shown in Fig. 2.

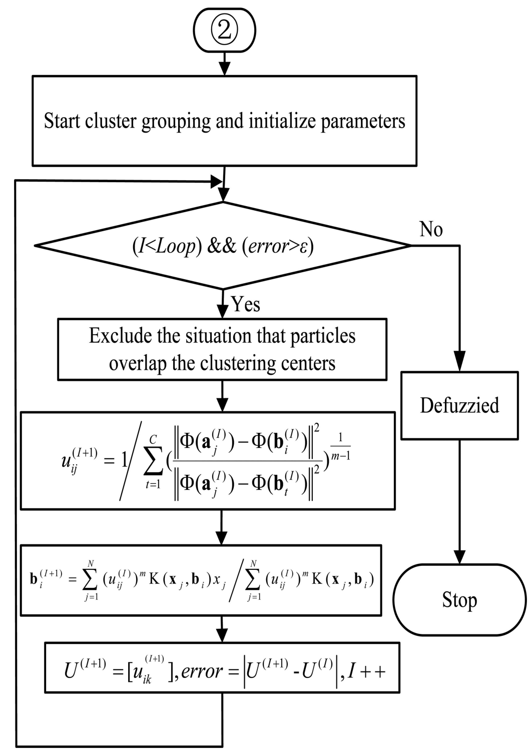


Fig. 2. Flowchart of the KFCM algorithm for CC grouping.

Thus, the high dimension in an optimization problem can be adaptively clustered into subgroups with lower dimensions. In the meantime, some similar dimensions are clustered into the same group by using the KFCM algorithm. In each group, these dimensions are incorporated into the search for the best position and share the information. Moreover, the population in one group can also be divided into several subswarms as discussed in the ensuing section.

D. Dynamic Neighbor Topology

Dynamic neighborhood topology (DNT) is introduced to PSO (PSO-DNT), which could assist the particles in sharing the information from the neighborhood particles, and separate the whole swarm into subswarms adaptively without decomposing.

Definition 1: The neighborhood diversity is measured by the attractive and repulsive optimizer, which is expressed as

$$\text{diversity}(S) = \frac{1}{S * L} \sum_{i=1}^{S} \sqrt{\sum_{d=1}^{D} (x_{id} - l_{gd})^2} \tag{15}$$

where S is the number of particles in one neighborhood, and L denotes the longest diagonal Euclidean distance at the search space. x_{id} and l_{gd} are particles' current positions and best positions in the same neighborhood, respectively. Through the relationship between the neighborhoods of all particles relative to optimal position, the diversity about the group particles is measured. It is not only playing a guiding role of the search for particles but also avoiding the issues caused by the

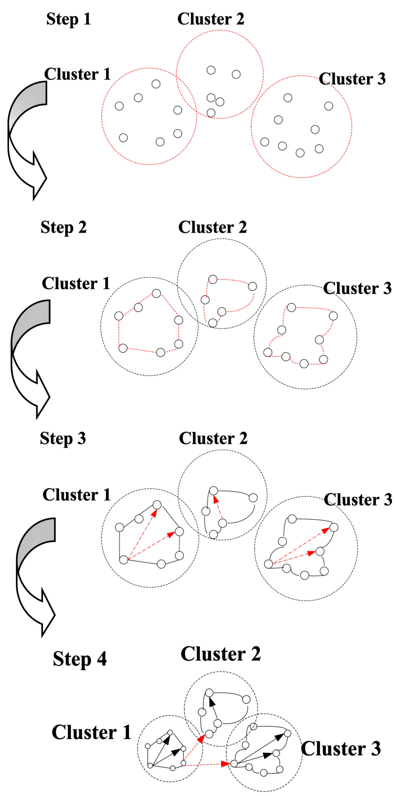


Fig. 3. Dynamic neighborhood topology schematic diagram.

increase in the number of calculations volume. Therefore, the main principle is that as the particles continue to search, each neighborhood of the diversity is detected in turn. If the particles are too concentrated in the attachment, and resulting in low diversity, as $\mathrm{diversity}(S) < d_{\min}$, then update the structure of neighborhood.

Now, the whole neighborhood topology is built up by the following step. First, according the clustering algorithm, classify the whole population into C groups. Then, establish the ring topology, in which each particle is connected with two neighbors in one group. After that, keep the original side in the network unchanged; randomly choose the node j corresponding to the node i to build up new topology relationship with the small probability p, which stands for the probability on the establishment of long-distance links. The construction process is shown in Fig. 3.

As shown in Fig. 3, 20 particles are clustered into three clusters in Step 1. After that, the ring topology is built up, in which each node connects to exactly two other nodes, forming a single continuous pathway for signals through each node—a ring. Then, long-distance links are obtained through the random connection in Step 3. By now, the PSO-DNT algorithm could optimize based on each subswarm neighborhood relationship, which would slow down convergence speed and increase diversity to achieve better results on multimodal problems.

The population is divided into small sized swarms. Each subswarm uses its own members to search for better regions in the search space. Therefore, the proposed dynamic neighborhood relationship is a new topology between the random connections and rule ones. The velocity update equation based on the neighborhood particles best position is presented as follow:

$$\begin{cases} v_{id}(k+1) = wv_{id}(k) + c_1\,\mathrm{rand}_{1d}()[p_{id}(k) - x_{id}(k)] \\ \quad + c_2\,\mathrm{rand}_{2d}()[l_{gd}(k) - x_{id}(k)] \\ x(k+1) = x(k) + v(k+1) \end{cases} \tag{16}$$

where $l_{gd}(k)$ is the best position in the neighborhood particles.

During this process, they may convergence to nearby local optimal and continually detect the neighborhood diversity $\mathrm{diversity}(S)$; if $\mathrm{diversity}(S) < \mathrm{d}_{\min}$, then the subswarms have already lost the optimization capability. Thus, renew the neighborhood connection among the different clusters like step 4 according to (4). The given probability ϑ of the neighborhood generation in the whole swarm could be determined by cross validation. The new swarms begin their search. This process is continued until a stop criterion satisfied. With a new randomly regrouped configuration, each small swarms search space is enlarged and better solutions are possible to be found by the new small swarms. The pseudocode of part ③ (see Fig. 1) of the PSO-DNT algorithm is shown as follows.

Algorithm Particle Swarm Optimization with Dynamic Neighbor Topology

Initialize the swarm and cluster the whole swarm into C clusters using KFCM;
Establish the ring topology in each cluster;
Build up long-distance topology in the same cluster through the random generation;
Repeat
1. Calculate the neighborhood best position l_{gd} and each particle best position p_{id};
2. Update the velocity v_{id} and position x_{id} using (16) and limit to the threshold range;
3. Calculate the neighborhood diversity using (15);
If $diversity(S) < d_{\min}$
Build up new connections in different clusters using (4);
End If
Until $\|x(k) - x(k-1)\| < \varepsilon$ or $k = TI_1$

IV. Results

In order to substantiate the effectiveness of the proposed FT-DNPSO algorithm for large-scale optimization, eight benchmark functions in [18], the new suite of test functions provided by the CEC2010 special session on large-scale global optimization and the functions of soft computing special session on large-scale computation are adopted to test. In order to overcome the fact that different initialization methods have different impacts on performance evaluation, the particle positions are initialized by a standard uniform distributed pseudorandom number generator in all experiments.

A. *Classical Benchmark Function Tests*

The eight classical functions are shown as follows:

Function 1: Sphere Function

$$f_1(x) = \sum_{i=1}^{n} x_i^2. \tag{17}$$

Function 2: Rosenbrock's Function

$$f_2(x) = \sum_{i=1}^{D-1} (100(x_i^2 - x_{i+1})^2 + (x_i - 1)^2). \quad (18)$$

Function 3: Ackley's Function

$$f_3(x) = -20 \exp\left(-0.2\sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2}\right) - \exp\left(\frac{1}{D}\sum_{i=1}^{D}\cos(2\pi x_i)\right) + 20 + e. \quad (19)$$

Function 4: Griewanks's Function

$$f_4(x) = \frac{1}{4000}\sum_{i=1}^{n} x_i^2 - \prod_{i=1}^{n}\cos\left(\frac{x_i}{\sqrt{i}}\right) + 1. \quad (20)$$

Function 5: Rastrigin Function

$$f_5(x) = \sum_{i=1}^{D} (x_i^2 - 10\cos(2\pi x_i) + 10). \quad (21)$$

Function 6: Weierstrass Function

$$f_6(x) = \sum_{i=1}^{D}\left(\sum_{k=0}^{k\max}[a^k \cos(2\pi b^k (x_i + 0.5))]\right) - D\sum_{k=0}^{k\max}[a^k \cos(2\pi b^k (x_i + 0.5))]$$
$$a = 0.5, b = 3, k\max = 20. \quad (22)$$

Function 7. Noncontinuous Rastrigin's Function

$$f_7(x) = \sum_{i=1}^{D}(y_i^2 - 10\cos(2\pi y_i) + 10)$$
$$y_i = \begin{cases} x_i & |x_i| < \frac{1}{2} \\ \frac{\text{round}(2x_i)}{2} & |x_i| >= \frac{1}{2} \end{cases} \quad \text{for} \quad i = 1, 2, \ldots, D. \quad (23)$$

Function 8: Schwefel's Function

$$f_8(x) = 418.9829 * D - \sum_{i=1}^{D} x_i \sin\left(|x_i|^{\frac{1}{2}}\right). \quad (24)$$

There are unimodal functions, multimodal functions, and discontinuous functions in these eight test functions. Hence, all different kinds of issues are considered [38]. Here, function 2 is chosen as an example to explain the "curse of dimensionality" phenomenon when the standard PSO (SPSO) algorithm deals with the large-scale optimization problem. There are similar characters for other functions, which are not shown repeated. In Fig. 4, the optimization behaviors of SPSO are depicted with the dimensions $D = 30$, $D = 100$, and $D = 1000$. During 3000 iterations, SPSO could obtain the best optimal result and demonstrate a well sustained search capability when $D = 30$. However, with the increase of dimension, the optimization process is increasingly difficult, and the reduction rate of objective function decreases. When $D = 1000$, SPSO basically loses its optimization ability. Therefore, the FT-DNPSO algorithm is proposed for large-scale optimization.

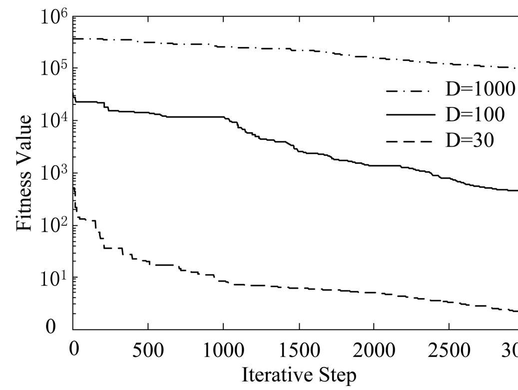


Fig. 4. Schematic diagram of "dimension curse" based on function 2.

TABLE I
EXPERIMENTAL CONDITIONS OF EIGHT TEST FUNCTIONS

f	x^*	$f(x^*)$	Search range
f_1	[0,0,⋯,0]	0	[-100,100]
f_2	[1,1,⋯,1]	0	[-2.048,2.048]
f_3	[0,0,⋯,0]	0	[-32.768,32.768]
f_4	[0,0,⋯,0]	0	[-600,600]
f_5	[0,0,⋯,0]	0	[-0.5,0.5]
f_6	[0,0,⋯,0]	0	[-5.12,5.12]
f_7	[0,0,⋯,0]	0	[-5.12,5.12]
f_8	[0,0,⋯,0]	0	[-500,500]

In order to compare the different versions of PSO, the experiment as conditions are shown in Table I. Tables II–IV summarize the results, which are the average (of the average fitness value in the swarm) calculated from all 30 times. For each benchmark test function, the dimensions are chosen as 30, 100, and 1000, respectively. And the size of swarm is set as 20, 20, and 30. The wPSO [10] with inertia weight, the PSO with ring topology neighborhood (NPSO) [26], the PSO with random topology neighborhood (RPSO) [30], and single PSO-DNT algorithm, DECC-G method [14] introduced by Yao, comprehensive learning PSO proposed by Suganthan termed as CLPSO [39], the proposed CC FT-DNPSO algorithm herein are evaluated. Best results are shown in bold face.

PC Configuration:

System: Windows XP SP 3; CPU: 3.00GHz; RAM: 2.00GB; Soft environment: MATLAB 2009a

As shown in the results, the FT-DNPSO achieves the best results on most high-dimensional complex multimodal problems. It implies that the proposed method could benefit from the FCM method and trust region algorithm. FCM cloud cluster the more approximate dimensions into one group in CC framework, and also cluster different particles into one subswarm in PSO-DNT, respectively. TR method adjusts the variable range according to the convergence process, which decreases the iterative times and explore in efficient neighborhood. PSO-DNT can learn from the neighborhood and the dynamic topology relationship, and make the information share better, avoiding

TABLE II
COMPARISON OF OPTIMIZATION RESULTS WITH DIMENSION $D = 30$

f	wPSO[10]	NPSO[26]	RPSO[30]	PSO-DNT	DECC-G[14]	CLPSO[39]	FT-DNPSO
f_1	9.78e-030	2.98e-073	9.39e-006	4.46e-014	1.82e-089	4.46e-014	**4.27e-110**
f_2	2.93e+001	7.09e+000	2.41e-001	**2.56e+000**	9.64e+001	2.10e+001	6.76e+001
f_3	3.94e-014	2.49e-014	5.92e-004	7.48e-030	8.62e-014	**0**	2.49e-014
f_4	8.13e-003	2.56e-002	2.57e-003	4.21e-001	5.38e-002	3.14e-010	**1.35e-011**
f_5	1.30e-004	**4.65e-015**	6.81e+001	6.96e-006	1.99e+000	4.85e-010	7.51e-007
f_6	2.90e+001	2.69e+001	9.41e+001	7.00e+000	1.55e-005	**3.45e-007**	5.55e-002
f_7	2.97e+001	1.36e-001	9.62e+002	6.15e-009	**0**	4.36e-010	4.75e-008
f_8	1.10e+003	2.33e+002	3.06e-002	1.85e-008	2.37e+002	**1.27e-012**	1.42e-010

TABLE III
COMPARISON OF OPTIMIZATION RESULTS WITH DIMENSION $D = 100$

f	wPSO[10]	NPSO[26]	RPSO[30]	PSO-DNT	DECC-G[14]	CLPSO[39]	FT-DNPSO
f_1	1.61e+003	1.65e+003	1.39e+003	1.31e+003	9.42e-071	3.28e-037	**3.11e-079**
f_2	4.00e+002	2.16e+001	4.43e+002	**3.60e+001**	1.57e+002	9.51e+001	**1.99e+002**
f_3	8.00e+001	1.60e+001	6.35e+000	2.79e+000	**2.22e-014**	3.19e-014	2.80e-008
f_4	1.60e+003	1.40e+001	1.55e+001	1.16e+000	3.12e-016	1.11e-002	**3.44e-019**
f_5	2.84e+002	5.14e+002	7.83e+002	6.35e+002	1.41e+001	1.78e+002	**4.04e+000**
f_6	3.32e+002	5.82e+002	8.00e+002	7.82e+001	**7.68e+000**	3.55e+002	7.46e+001
f_7	2.87e+003	2.19e+004	2.39e+004	3.75e+003	1.83e-009	**2.84e-014**	1.45e-011
f_8	2.63e+001	1.03e+002	3.26e+001	2.85e-004	1.09e-010	1.00e+000	**2.68e-014**

TABLE IV
COMPARISON OF OPTIMIZATION RESULTS WITH DIMENSION $D = 1000$

f	wPSO[10]	NPSO[26]	RPSO[30]	PSO-DNT	DECC-G[14]	CLPSO[39]	FT-DNPSO
f_1	9.81e+004	8.06e+005	1.43e+005	2.56e+004	2.17e-025	5.29e-001	**1.18e-029**
f_2	1.47e+004	1.21e+005	1.45e+004	4.34e+003	9.87e+002	9.98e+002	**3.39e+001**
f_3	1.96e+001	1.87e+001	1.95e+001	1.72e+001	**2.22e-013**	4.38e-002	1.35e-004
f_4	8.26e+002	7.09e+003	1.19e+003	2.79e+002	1.01e-015	3.45e-002	**4.41e-016**
f_5	6.99e+003	1.37e+004	7.16e+003	3.76e+003	3.55e+002	3.32e+002	**1.27e+001**
f_6	1.38e+005	1.37e+004	7.86e+003	5.01e+003	3.12e+002	2.06e+003	**1.00e+002**
f_7	7.55e+002	3.68e+005	1.37e+005	1.58e+005	**8.70e+000**	5.32e+002	3.85e+001
f_8	8.58e+003	1.35e+003	7.98e+002	7.73e+002	2.13e+002	2.04e+004	**3.51e+001**

lapsing into the degeneracy. It is seen that CLPSO [39] achieves the best results when the dimension is low. When the dimension increases, DECC-G [14] and the proposed FT-DNPSO methods are more superior. FT-DNPSO introduces dynamic neighborhood to large-scale optimization, so as to ensure the algorithm to determinate the maximize space and eventually obtain a better accuracy when $D = 1000$.

B. CEC2010 Special Session Test

Furthermore, 20 CEC'10 test functions are considered in the experiment. Here is the summary of the 20 test problems [40].

1) *Separable functions:*
   a) F1: Shifted Elliptic Function.
   b) F2: Shifted Rastrigin's Function.
   c) F3: Shifted Ackley's Function.
2) *Single-group m-nonseparable functions:*
   a) F4: Single-group Shifted and m-rotated Elliptic Function.
   b) F5: Single-group Shifted and m-rotated Rastrigin's.
   c) F6: Single-group Shifted and m-rotated Ackley's.
   d) F7: Single-group Shifted m-dimensional Schwefel's 1.2
   e) F8: Single-group Shifted m-dimensional Rosenbrock's.

TABLE V
EXPERIMENTAL CONDITIONS OF CEC 2010

Parameter names	Parameter values
Dimension	1000
Group	20
Training times	3000000
Independent group training times	2000
Swarm size	100
Acceleration coefficient	c_1=1.49, c_2=1.49
Inertia weight	$\omega = 0.8$

3) *D/2 m-group m-nonseparable functions:*
   a) F9: D/2 m-group Shifted and m-rotated Elliptic Function.
   b) F10: D/2 m-group Shifted and m-rotated Rastrigin's.
   c) F11: D/2 m-group Shifted and m-rotated Ackley's.
   d) F12: D/2 m-group Shifted m-dimensional Schwefel's 1.2.
   e) F13: D/2 m-group Shifted m-dimensional Rosenbrock's.
4) *D/m-group m-nonseparable functions:*
   a) F14: D/m-group Shifted and m-rotated Elliptic Function.
   b) F15: D/m-group Shifted and m-rotated Rastrigin's.

TABLE VI
RANKS OF ACCURACY COMPARISON AMONG FOUR ALGORITHMS

	Func.1	Func.2	Func.3	Func.4	Func.5
DECC-G [14]	2.93e-007	1.31e+003	1.39e+000	1.70e+013	2.63e+008
DECC-DML [22]	**1.93e-025**	2.17e+002	1.18e-013	3.58e+012	2.99e+008
DMS-PSO-SHS [23]	5.51e-015	8.51e+001	5.52e-011	2.46e+011	8.36e+007
FT-DNPSO	4.13e-023	**8.23e+001**	**1.71e-015**	**7.59e+010**	**2.78e+007**
	Func.6	Func.7	Func.8	Func.9	Func.10
DECC-G [14]	4.96e+006	1.63e+008	6.44e+007	3.21e+008	1.06e+004
DECC-DML [22]	7.93e+005	1.39e+008	3.46e+007	5.92e+007	1.25e+004
DMS-PSO-SHS [23]	**8.28e-002**	1.95e+003	1.29e+007	8.72e+006	5.53e+003
FT-DNPSO	3.67e+006	**5.56e+001**	**1.13e+006**	**1.26e+006**	**3.78e+003**
	Func.11	Func.12	Func.13	Func.14	Func.15
DECC-G [14]	2.34e+001	8.93e+004	5.12e+003	8.08e+008	1.22e+004
DECC-DML [22]	**1.80e-013**	3.80e+006	1.14e+003	1.89e+008	1.54e+004
DMS-PSO-SHS [23]	3.25e+001	6.13e+002	1.12e+003	**1.76e+007**	4.08e+003
FT-DNPSO	2.03e+002	**4.76e+002**	**6.38e+002**	1.85e+008	**1.88e+003**
	Func.16	Func.17	Func.18	Func.19	Func.20
DECC-G [14]	7.66e+001	2.87e+005	2.46e+004	1.11e+006	4.06e+003
DECC-DML [22]	**5.08e-002**	6.54e+006	2.47e+003	1.59e+007	9.91e+002
DMS-PSO-SHS [23]	6.98e+001	**3.83e+003**	2.26e+003	1.17e+006	3.52e+002
FT-DNPSO	4.27e+001	2.31e+004	**2.09e+003**	**1.04e+006**	**2.39e+002**

c) F16: D/m-group Shifted and m-rotated Ackley's.
d) F17: D/m-group Shifted m-dimensional Schwefel's1.2.
e) F18: D/m-group Shifted m-dimensional Rosenbrock's.

5) *Nonseparable functions:*
a) F19: Shifted Schwefel's Problem 1.2.
b) F20: Shifted Rosenbrock's Function.

The experiments for each function are repeated 20 times. The experiment condition and mean results are tabulated in Tables V and VI. DECC-G [14], DECC-DML [22], and DMS-PSO-SHS [23] are employed to compare with FT-DNPSO. All of the results for these three methods are recorded directly from the published literature. Best results are shown in bold face. The convergence plots of FT-DNPSO are shown in Fig. 5.

As shown in Table VI, the proposed FT-DNPSO obtains the best accuracy in 14 functions. The other six results are close to the best one. FT-DNPSO could not achieve best results for functions 1, 6, 11, and 16, which are different Ackley versions, because the local minimums of Ackley functions. It is also seen that FT-DNPSO falls into the local minimum from Fig. 5. However, it could obtain the continuous search capability for the other functions.

Then, statistical techniques are employed to analyze the evolutionary algorithm's behavior over optimization problems. In the multiple-problem analysis, due to the dissimilarities in the results obtained and the small size of the sample to be analyzed, a parametric test may reach an erroneous conclusion. Therefore, the nonparametric statistics method proposed by Garcia [41] is adopted for comparing the aforementioned results. The results of the Friedman test are tabulated in Table VII.

On average, FT-DNPSO ranks first with rank 1.45; DMS-PSO-SHS [23] and DECC-DML [22] rank second and the third, respectively; and the last is DECC-G with rank 3.55. Under the null hypothesis, which states that all the algorithms behave similarly, and thus, their rank should be equal, the Friedman statistic [41] is

$$\chi_F^2 = \frac{12n}{k(k+1)}\left[\sum_j R_j^2 - \frac{k(k+1)^2}{4}\right] = 30.3 \tag{25}$$

TABLE VII
EXPERIMENT RESULTS OF CEC 2010

	DECC-G [14]	DECC-DML [22]	DMS-PSO-SHS [23]	FT-DNPSO
Average rank	3.55	2.90	2.10	1.45

with $p_{\text{value}} = 1.1934 \times 10^{-6}$ and three degrees of freedom, here, $R_j = \frac{1}{n}\sum_i r_i^j$ and r_i^j is the rank of the jth of k algorithms on the ith of n datasets. And the Iman-Davenport statistic [41] is

$$F_F = \frac{(n-1)\chi_F^2}{n(k-1)-\chi_F^2} = 16.1039 \tag{26}$$

with $p_{\text{value}} = 1.06315 \times 10^{-7}$ and (3, 57) degrees of freedom, therefore, the null hypothesis is rejected at a high level of significance.

C. Soft Computing Special Session Test

A set of 19 scalable function optimization problems is provided by soft computing special session [40]. There are six functions of the CEC2008 test suite, five shifted functions, and eight hybrid composition functions. The details of the set of scalable functions are available at http://sci2 s.ugr.es/EAMHCO /testfunctions-SOCO.pdf. Each algorithm is run 25 times for each test function. The maximum number of fitness evaluations was 5 000 000. The problem dimension is 1000, and the swarm size is 100. Each run stops when the maximal number of evaluations is achieved. All of test parameters are set according to [42]. The basic algorithm DE [43] and CHC [44] are employed to this simulation. On the other hand, the SaDE-MMTS [45] and other PSO versions, such as RPSO-vm [46] and Tuned IPSOLS [47] are also adopted. The experiment results are tabulated in Table VIII. All of the results for these five methods are recorded directly from the published literature. Best results are shown in bold face. From the comparison results, the proposed FT-DNPSO obtains the best results in the 13 functions, as well as acquires the better results in the other six functions like Func.2 and Func.13.

Then, the nonparametric statistics of the Friedman test [41] are adopted to analyze the results as tabulated in Table IX.

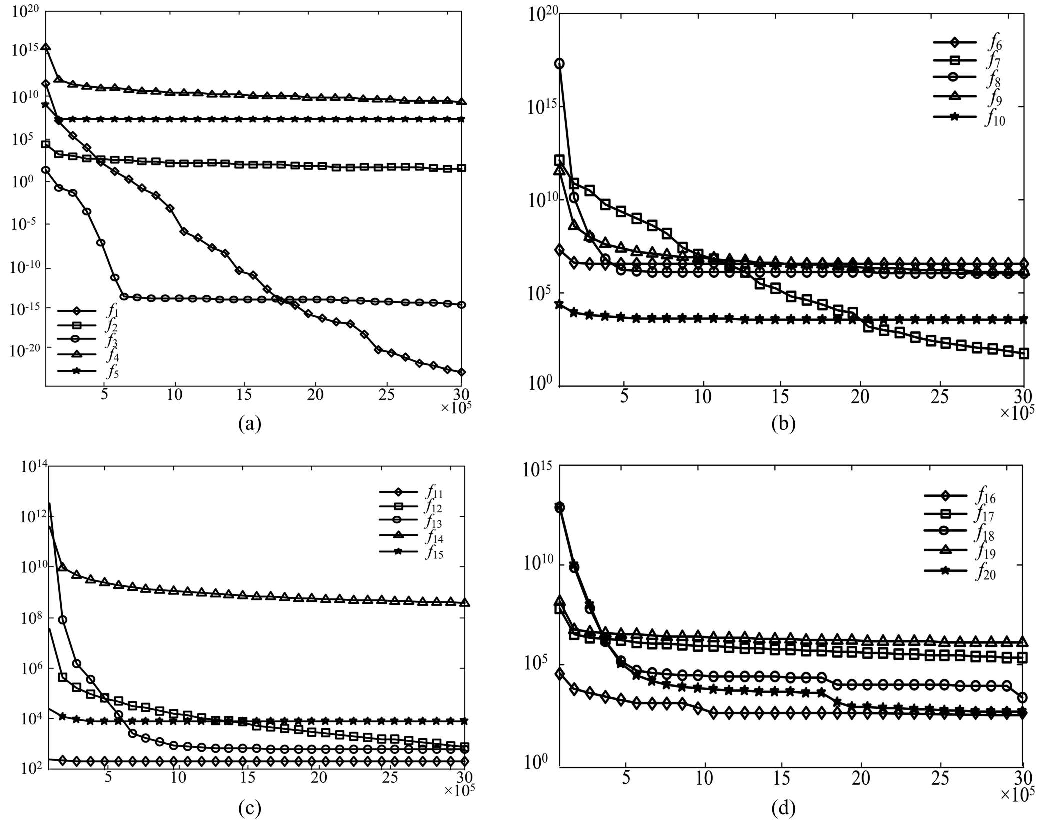


Fig. 5. Convergence of 20 test functions for the proposed FT-DNPSO algorithm. (a) Functions 1–5. (b) Functions 6–10. (c) Functions 11–15. (d) Functions 16–20.

TABLE VIII
EXPERIMENT RESULTS OF SOFT COMPUTING SPECIAL SESSION

	Func.1	Func.2	Func.3	Func.4	Func.5
DE [43]	**0.00e+00**	8.46e+01	9.69e+02	1.44e+00	**0.00e+00**
CHC [44]	1.36e-11	1.44e+02	8.75e+03	4.76e+03	7.02e-03
SaDE-MMTS [45]	**0.00e+00**	4.88e+01	**0.00e+00**	3.21e+01	**0.00e+00**
RPSO-vm [46]	2.72e-14	4.29e+01	3.21e+02	4.81e-14	2.14e-01
Tuned IPSOLS [47]	**0.00e+00**	**6.68e-14**	**0.00e+00**	**0.00e+00**	**0.00e+00**
FT-DNPSO	**0.00e+00**	3.73e+01	2.07e+02	**0.00e+00**	**0.00e+00**
	Func.6	Func.7	Func.8	Func.9	Func.10
DE [43]	3.29e-12	**0.00e+00**	2.46e+05	5.13e+03	**0.00e+00**
CHC [44]	1.38e+01	3.52e-01	3.11e+05	6.11e+03	3.83e+02
SaDE-MMTS [45]	**0.00e+00**	**0.00e+00**	1.46e+03	9.25e+01	**0.00e+00**
RPSO-vm [46]	4.92e-12	3.63e-15	9.35e+05	1.17e+01	**0.00e+00**
Tuned IPSOLS [47]	**0.00e+00**	7.13e-11	1.20e+05	9.76e+00	**0.00e+00**
FT-DNPSO	**0.00e+00**	**0.00e+00**	**3.03e+02**	**1.98e+00**	**0.00e+00**
	Func.11	Func.12	Func.13	Func.14	Func.15
DE [43]	1.35e-03	1.68e-08	7.30e+02	6.90e-01	0.00e+00
CHC [44]	4.82e+03	1.05e+03	6.66e+07	3.62e+03	8.31e+01
SaDE-MMTS [45]	1.85e+02	**0.00e+00**	6.55e+02	1.40e+02	**0.00e+00**
RPSO-vm [46]	1.10e+01	1.00e-09	1.93e+03	**5.27e-01**	**0.00e+00**
Tuned IPSOLS [47]	**9.75e+00**	1.37e+00	**1.28e+00**	1.78e+01	1.39e-10
FT-DNPSO	2.13e+02	**0.00e+00**	1.68e+01	4.41e+02	**0.00e+00**
	Func.16	Func.17	Func.18	Func.19	
DE [43]	4.18e-08	2.36e+02	2.37e-03	**0.00e+00**	
CHC [44]	2.32e+03	2.04e+07	1.72e+03	4.20e+03	
SaDE-MMTS [45]	**0.00e+00**	1.90e+02	8.20e+01	**0.00e+00**	
RPSO-vm [46]	9.50e-01	2.82e+03	**1.80e+00**	**0.00e+00**	
Tuned IPSOLS [47]	2.41e+00	6.49e+00	2.46e+01	1.01e-10	
FT-DNPSO	**0.00e+00**	**6.03e+00**	2.12e+02	**0.00e+00**	

On average, FT-DNPSO ranks first with rank 2.447; the last is CEC [44] with rank 5.737; SaDE-MMTS [45] and Tuned IPSOLS [47] have the DMS-PSO-SHS [23] are near the FT-DNPSO. Under the null hypothesis, which states that all the algorithms behave similarly, and thus, their rank should be equal, the Friedman statistic [41] is

$$\chi_F^2 = \frac{12n}{k(k+1)}\left[\sum_j R_j^2 - \frac{k(k+1)^2}{4}\right] = 38.0922 \quad (27)$$

TABLE IX
RANKS OF ACCURACY COMPARISON AMONG THE SIX ALGORITHMS

	FT-DNPSO	DE [43]	CHC [44]
Average rank	2.447368	3.263158	5.736842
	SaDE-MMTS [45]	RPSO-vm [46]	Tuned IPSOLS [47]
Average rank	2.868421	3.789474	2.894737

with $p_{\rm value} = 3.6190 \times 10^{-7}$ and five degrees of freedom. And the Iman–Davenport statistic [41] is

$$F_F = \frac{(n-1)\chi_F^2}{n(k-1) - \chi_F^2} = 12.0476 \tag{28}$$

with $p_{\rm value} = 6.10203 \times 10^{-9}$ and (5, 90) degrees of freedom, therefore, the null hypothesis is rejected at a high level of significance. Hence, statistical analysis proves that FT-DNPSO could also be superior in the soft computing special session.

V. CONCLUSION

In this paper, a novel FT-DNPSO algorithm is proposed. The FT-DNPSO combines FCM clustering and the trust region algorithm with CC strategy for large-scale optimization. It adaptively adjusts the initial region and clusters different dimension into groups, which expedites convergence and search in the effective range. At each group, the PSO-DNT algorithm with dynamic neighborhood topology and multiswarm search is utilized to tackle the original stable structure for the whole optimization process. The adaptive strategy avoids or alleviates the prematurity of the PSO algorithm. The simulation results, with eight classical benchmark functions, twenty CEC2010 test ones, and soft computing special session test demonstrate that the proposed FT-DNPSO outperformed other PSO algorithms for large-scale optimization. However, all problems considered in this study are unconstrained optimization. Future work may include the extension of the proposed method for large-scale constraint optimization.

ACKNOWLEDGMENT

The authors would like to thank P. N. Suganthan of Nanyang Technological University and Z. Yang of the University of Science and Technology of China for providing many versions of metaheuristic optimization algorithm codes and benchmark function materials. They would also like to thank S. Garcia of the University of Jaén for providing the nonparameter statistic method to analyze the behaviors of evolutionary algorithms.

REFERENCES

[1] P. Kenneth and S. Rainer, *Differential Evolution*. Berlin, Germany: Springer, 1997.

[2] S. Das and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art," *IEEE Trans. Evol. Comput.*, vol. 15, no. 1, pp. 4–31, Feb. 2011.

[3] D. Marco, "Optimization, learning and natural algorithm," Ph.D. dissertation, Syst. Inf. Eng., Politecnico di Milano, Italy, 1992.

[4] Y. R. Zhou, "Runtime analysis of an ant colony optimization algorithm for TSP Instances," *IEEE Trans. Evol. Comput.*, vol. 13, no. 5, pp. 1083–1092, Oct. 2009.

[5] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Netw.*, 1995, vol. 4, pp. 1942–1948.

[6] R. P. Prado, S. G. Garcia, J. E. M. Exposito, and A. J. Yuste, "Knowledge acquisition in fuzzy-fule-based systems with particle-swarm optimization," *IEEE Trans. Fuzzy Syst.*, vol. 18, no. 6, pp. 1083–1097, Dec. 2010.

[7] G. G. Yen and W. F. Leong, "Dynamic multiple swarms in multiobjective particle swarm optimization," *IEEE Trans. Syst., Man, Cybern. A, Syst., Humans*, vol. 39, no. 4, pp. 890–911, Jul. 2009.

[8] P. Larranaga and J. A. Lozano, *Estimation of Distribution Algorithm: A New Tool for Evolutionary Computation*. New York, NY, USA: Springer, 2002, vol. 2.

[9] J. Kennedy and R. Mendes, "Neighborhood topologies in fully informed and best-of-neighborhood particle swarms," *IEEE Trans. Syst., Man, Cybern. C, Appl., Rev.*, vol. 36, no. 4, pp. 515–519, Jul. 2006.

[10] P. J. Angeline, "Using selection to improve particle swarm optimization," in *Proc. IEEE World Congr. Comput. Intell.*, 1998, pp. 84–89.

[11] R. J. Wai, K. L. Chuang, and J. D. Lee, "On-line supervisory control design for maglev transportation system via total sliding-mode approach and particle swarm optimization," *IEEE Trans. Autom. Control*, vol. 55, no. 7, pp. 1544–1559, Jul. 2010.

[12] J. Hazra and A. K. Sinha, "Congestion management using multiobjective particle swarm optimization," *IEEE Trans. Power Syst.*, vol. 22, no. 4, pp. 1726–1734, Nov. 2007.

[13] J. Q. Wang and H. Y. Zhang, "Multicriteria decision-making approach based on Atanassov's intuitionistic fuzzy sets with incomplete certain information on weights," *IEEE Trans. Fuzzy Syst.*, vol. 21, no. 3, pp. 510–515, Jun. 2013.

[14] Z. Y. Yang, K. Tang, and X. Yao, "Large scale evolutionary optimization using cooperative coevolution," *Inf. Sci.*, vol. 178, no. 15, pp. 2985–2999, 2008.

[15] S. K. S. Fan and J. M. Chang, "Dynamic multi-swarm particle swarm optimizer using parallel PC cluster systems for global optimization of large-scale multimodal functions," *Eng. Optim.*, vol. 42, no. 5, pp. 431–451, 2010.

[16] K. Scheerlinck, H. Vernieuwe, and B. De Baets, "Zadeh's extension principle for continuous functions of non-interactive variables: A parallel optimization approach," *IEEE Trans. Fuzzy Syst.*, vol. 20, no. 1, pp. 96–108, Feb. 2012.

[17] T. Korenaga, T. Hatanaka, and K. Uosaki, "Improvement of particle swarm optimization for high-dimensional space," in *Proc. SICE-ICASE Int. Joint Conf.*, 2006, pp. 2174–2178.

[18] F. Van den Bergh and A. P. Engelbrecht, "A cooperative approach to particle swarm optimization," *IEEE Trans. Evol. Comput.*, vol. 8, no. 3, pp. 225–239, Jun. 2004.

[19] J. J. Liang and P. N. Suganthan, "Dynamic multi-swarm particle swarm optimizer with a novel constraint-handling mechanism," in *Proc. IEEE Congr. Evol. Comput.*, 2006, pp. 9–16.

[20] P. Subbaraj, R. Rengaraj, S. Salivahanan, and T. R. Senthilkumar, "Parallel particle swarm optimization with modified stochastic acceleration factors for solving large scale economic dispatch problem," *Int. J. Elect. Power Energy Syst.*, vol. 32, no. 9, pp. 1014–1023, 2010.

[21] M. N. Omidvar, X. D. Liu, X. Yao, and Z. Y. Yang, "Cooperative Co-evolution for large scale optimization through more frequent random grouping," in *Proc. IEEE Congr. Evol. Comput.*, 2010, pp. 1–8.

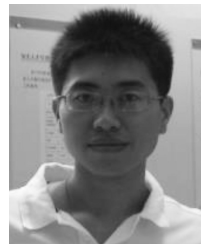
[22] M. N. Omidvar, X. D. Li, and X. Yao, "Cooperative co-evolution with delta grouping for large scale non-separable function optimization," in *Proc. IEEE Congr. Evol. Comput.*, 2010, pp. 1–8.

[23] S. Z. Zhao, P. N. Suganthan, and S. Das, "Dynamic multi-swarm particle swarm optimizer with sub-regional harmony search," in *Proc. IEEE Congr. Evol.Comput.*, 2010, pp. 1983–1990.

[24] R. Mallipeddi and P. N. Suganthan, "Differential evolution with ensemble of constraint handling techniques for solving CEC 2010 benchmark problems," in *Proc. IEEE Congr. Evol. Comput.*, 2010, pp. 1–8.

[25] Y. G. Fu, M. Y. Ding, and C. P. Zhou, "Phase angle-encoded and quantum-behaved particle swarm optimization applied to three-dimensional route planning for UAV," *IEEE Trans. Syst., Man, Cybern. A, Syst., Humans*, vol. 42, no. 2, pp. 511–526, Mar. 2012.

[26] J. Kennedy, "Small worlds and mega-minds: effects of neighborhood topology on particle swarm performance," in *Proc. IEEE Congr. Evol. Comput.*, 1999, pp. 1931–1938.
[27] A. M. Liu, X. L. Zhang, X. Lin, and Y. X. Li, "Application of neighborhood topology particle swarm optimization to cylinder linear induction motor design," in *Proc. IEEE Int. Conf. Autom. Logistics*, 2009, pp. 538–542.
[28] Y. P. Chen, W. C. Peng, and M. C. Jian, "Particle swarm optimization with recombination and dynamic linkage discovery," *IEEE Trans. Syst., Man, Cybern. B , Cybern.*, vol. 37, no. 6, pp. 1460–1470, Dec. 2007.
[29] J. Lane, A. Engelbrecht, and J. Gain, "Particle swarm optimization with spatially meaningful neighbors," in *Proc. IEEE Swarm Intell. Symp.*, 2008, pp. 1–8.
[30] S. B. Akat and V. Gazi, "Particle swarm optimization with dynamic neighborhood topology: Three neighborhood strategies and preliminary results," in *Proc. IEEE Swarm Intell. Symp.*, 2008, pp. 1–8.
[31] A. Seetharam, A. Bhattacharyya, and M. K. Naskar, "A position based algorithm for topology management in mobile Ad-Hoc networks enduring limited transmission failures," in *Proc. Int. Conf. Inf. Commun. Technol. Electr. Sci.*, 2007, pp. 928–933.
[32] E. S. Peer, F. van den Bergh, and A. P. Engelbrecht, "Using neighborhoods with the guaranteed convergence PSO," in *Proc. IEEE Swarm Intell. Symp.*, 2003, pp. 235–242.
[33] W. Elshamy, H. M. Emara, and A. Bahgat, "Clubs-based particle swarm optimization," in *Proc. IEEE Swarm Intell. Symp.*, 2007, pp. 289–296.
[34] J. Kennedy and R. Mendes, "Population structure and particle swarm performance," in *Proc. 4th Congr. Evol. Comput.*, 2002, pp. 1671–1676.
[35] S. Cheng, Y. H. Shi, and Q. D. Qin, "Population diversity based study on search information propagation in particle swarm optimization," in *Proc. IEEE World Congr. Comput. Intell.*, 2012, pp. 1272–1279.
[36] E. G. Birgin, E. V. Castelani, A. L. M. Martinez, and J. M. Martinez, "Outer trust-region method for constrained optimization," *J. Optim. Theory Appl.*, vol. 150, no. 1, pp. 142–155, 2011.
[37] L. Chen, C. L. P. Chen, and M. Z Lu, "A multiple-kernel fuzzy C-means algorithm for image segmentation," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 41, no. 5, pp. 1263–1274, Oct. 2011.
[38] M. Han and J. C. Fan, "Particle swarm optimization using dynamic neighborhood topology for large scale optimization," in *Proc. 8th World Congr. Intell. Control Autom.*, 2010, pp. 3138–3142.
[39] J. J. Liang, A. K. Qin, P. N. Suganthan, and S. Baskar, "Comprehensive learning particle swarm optimizer for global optimization of multimodal functions," *IEEE Trans. Evol. Comput.*, vol. 10, no. 3, pp. 281–295, Jun. 2006.
[40] K. Tang, X. D. Li, P. N. Suganthan, Z. Y. Yang, and T. Weise. (2010). "Benchmark functions for the CEC2010 special session and competition on large-scale global optimization." [Online]. Available: http://nical.ustc.edu.cn/cec10 ss.php
[41] S. Garcia, A. Fernandez, J. Luengo, and F. Herrera, "Advanced nonparametric tests for multiple comparisons in the design of experimental in computational intelligence and data mining: Experimental analysis of power," *Inf. Sci.*, vol. 180, no. 10, pp. 2044–2064, 2010.
[42] M. Lozano, D. Molina, and F. Herrera, "Editorial scalability of evolutionary algorithms and other metaheuristics for large-scale continuous optimization problems," *Soft comput.*, vol. 15, no. 11, pp. 2085–2087, 2011.
[43] R. Storn and K. Price, "Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces," *J. Global Optim.*, vol. 11, no. 4, pp. 341–359, 1997.
[44] L. J. Eshelman and J. D. Schaffer, "Real-coded genetic algorithms and interval-schemata," in *Found.ations of Genetic Algorithms 2*, L. D. Whitley, Ed. San Mateo, CA, USA: Morgan Kaufmann, 1992, pp. 187–202.
[45] S. Z. Zhao, P. N. Suganthan, and S. Das, "Self-adaptive differential evolution with multi-trajectory search for large scale optimization," *Soft Comput.*, vol. 15, no. 11, pp. 2175–2185, 2011.
[46] G. N. José and A. Enrique, "Restart particle swarm optimization with velocity modulation: A scalability test," *Soft Comput.*, vol. 15, no. 11, pp. 2221–2232, 2011.
[47] A. Marco, A. Dogan, and S. Thomas, "An incremental particle swarm for large-scale optimization problems: An example of tuning-in-the-loop design of optimization algorithms," *Soft Comput.*, vol. 15, no. 11, pp. 2233–2255, 2011.



Jianchao Fan (S'12–M'13) received the M.S. and Ph.D. degrees from the School of Control Science and Engineering, Dalian University of Technology, Dalian, China, in 2009 and 2012, respectively.

He is a Research Associate with the Department of Ocean Remote Sensing, National Marine Environmental Monitoring Center, Dalian and Marine Environmental Protection Institute, State Oceanic Administration. His current research interests include fuzzy clustering, large-scale computation, particle swarm optimization, neural networks, and remote sensing processing.

Jun Wang (S'89–M'90–SM'93–F'07) received the B.S. degree in electrical engineering and the M.S. degree in systems engineering from the Dalian University of Technology, Dalian, China, in 1982 and 1985, respectively, and the Ph.D. degree in systems engineering from Case Western Reserve University, Cleveland, OH, USA, in 1991.

He is a Professor with the Department of Mechanical and Automation Engineering, The Chinese University of Hong Kong, Hong Kong. He held various academic positions with the Dalian University of Technology; Case Western Reserve University; and the University of North Dakota, Grand Forks, ND, USA. He also held various short-term or part-time visiting positions with the U.S. Air Force Armstrong Laboratory in 1995, the RIKEN Brain Science Institute in 2001, the Universite Catholique de Louvain in 2001, the Chinese Academy of Sciences in 2002, the Huazhong University of Science and Technology in 2006–2007, Shanghai Jiao Tong University as a Cheung Kong Chair Professor in 2008–2011, and has been the Dalian University of Technology as a National Thousand-Talent Chair Professor since 2011. His current research interests include neural networks and their applications.

Dr. Wang has been an Associate Editor of the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS—PART B since 2003, an Editorial Advisory Board Member of the *International Journal of Neural Systems* since 2006, and an Editorial Board Member of *Neural Networks* since 2012. He also served as an Associate Editor of the IEEE TRANSACTIONS ON NEURAL NETWORKS from 1999 to 2009 and the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS—PART C from 2002 to 2005. He was a Guest Editor of special issues of the *European Journal of Operational Research* in 1996, the *International Journal of Neural Systems* in 2007, and *Neurocomputing* in 2008 and 2013. He served as the President of the Asia Pacific Neural Network Assembly in 2006, the General Chair of the13th International Conference on Neural Information Processing in 2006, and the IEEE World Congress on Computational Intelligence in 2008. In addition, he has served on many technical and standing committees, such as the IEEE Fellow Committee. He has been a Fellow of the International Association of Pattern Recognition since 2012 and was an IEEE Computational Intelligence Society Distinguished Lecturer during 2010–2012. He received the Research Excellence Award from The Chinese University of Hong Kong for 2008–2009, two Natural Science Awards (first class), respectively, from the Shanghai Municipal Government in 2009 and the Ministry of Education of China in 2011, the Outstanding Achievement Award from Asia Pacific Neural Network Assembly, and the IEEE TRANSACTIONS ON NEURAL NETWORKS Outstanding Paper Award (with Q. Liu) from the IEEE Computational Intelligence Society in 2011.

Min Han (M'95–A'03–SM'06) received the B.S. and M.S. degrees from the Department of Electrical Engineering, Dalian University of Technology, Dalian, China, in 1982 and 1993, respectively, and the M.S. and Ph.D. degrees from Kyushu University, Fukuoka, Japan, in 1996 and 1999, respectively.

She is a Professor with the School of Electronic and Information Engineering, Dalian University of Technology. Her current research interests include neural network and chaos and their applications to control and identification.